BountyLine: Informe ejecutivo final

Universidad El Bosque · Ingeniería de Sistemas · Bases de datos 2

Gabriella Michelle Castro Cuello

Lina Sofía Patiño Calderón

Julian Samuel Rodriguez Hernandez

Bogotá D.C., noviembre de 2026

Este informe resume el proyecto final de Bases de Datos 2: el problema que se abordó, la solución construida, su arquitectura, las decisiones técnicas que la sostienen, los resultados obtenidos y lo que el equipo aprendió en el proceso. El detalle de cada parte está en los documentos de la Fase 1 y la Fase 2, que se listan como anexos al final.

1. Problema

BountyLine se ubica dentro de un videojuego de mundo abierto ambientado en una ciudad futurista. Allí los jugadores tienen conflictos entre ellos, como robos de créditos, deudas o zonas peligrosas que no pueden recorrer solos, y no existe un mecanismo confiable para pedir ayuda y pagar por ella. Todo depende de mensajes informales entre conocidos, no hay forma de saber qué jugador es confiable para cierto trabajo y nadie garantiza que la recompensa prometida llegue a quien hizo el trabajo, ni que se devuelva si la misión se cancela.

Desde el punto de vista de datos, el caso combina dos tipos de información muy distintos. Por un lado están los créditos, las misiones y las calificaciones, donde un error deja saldos inconsistentes y la integridad es obligatoria. Por otro lado está el contenido social, con publicaciones, comentarios, chat y registros de actividad, que crece rápido y cambia de forma con frecuencia. Esa mezcla es la que justifica usar una base de datos relacional con lógica de negocio y una base de datos documental al mismo tiempo.

2. Solución

BountyLine es una red social con mercado de misiones. Tiene tres tipos de cuenta. El Usuario publica en el feed, crea misiones con una recompensa en créditos, asigna a un cazarrecompensas, conversa con él por chat, confirma el cierre y lo califica. El Cazarrecompensas mantiene su perfil, su zona base y sus especialidades, busca misiones, se postula y recibe el pago. El Administrador gestiona los catálogos de zonas y especialidades, atiende reportes, oculta contenido y suspende cuentas.

El flujo central es el de una misión. Al crearla, la base de datos valida el saldo y retiene el monto en la billetera del usuario. El sistema sugiere cazarrecompensas con un puntaje que combina zona, especialidad y reputación, y el usuario también puede buscar con filtros. Cuando el usuario confirma el cierre, en una sola transacción se libera el monto retenido, se acredita al

cazarrecompensas y se registran los movimientos. Si la misión se cancela, el monto vuelve al usuario. Al final, la calificación actualiza la reputación de forma automática por medio de un trigger.

3. Arquitectura

La solución tiene tres capas, y las bases de datos se ejecutan en dos máquinas virtuales separadas dentro de VirtualBox.

Capa	Tecnología	Responsabilidad
Presentación	Angular con TypeScript	Interfaz web para los tres tipos de cuenta
Aplicación	Spring Boot 4 con Java 21	API REST. Llama procedimientos almacenados por JDBC y usa Spring Data MongoDB
Datos relacionales	MySQL 8.4 LTS en la VM BountyLine-MySQL (Ubuntu Server 26.04 LTS)	Cuentas, billeteras, misiones, postulaciones, movimientos, calificaciones, reportes y auditoría
Datos documentales	MongoDB 8.0 en la VM BountyLine-MongoDB (Ubuntu Server 26.04 LTS)	Publicaciones con comentarios embebidos, mensajes de chat y bitácora de actividad

El backend no escribe directamente en las tablas de MySQL. Todas las operaciones que cambian datos pasan por procedimientos almacenados, que validan las reglas de negocio y lanzan errores con SIGNAL cuando algo no se cumple. El backend traduce esos errores en respuestas claras para el cliente. Los documentos de MongoDB guardan los identificadores de cuenta y de misión de MySQL para relacionar el contenido social con los datos transaccionales sin duplicar el modelo relacional. Los diagramas C4 de contexto, contenedores y componentes están en la propuesta y en el anexo C4.

4. Decisiones técnicas

Decisión	Motivo
MySQL 8.4 LTS como motor relacional	Soporta procedimientos, funciones, triggers con SIGNAL, restricciones CHECK y transacciones con InnoDB. Es gratuito, tiene soporte de largo plazo y se conecta a Spring Boot con mysql-connector-j. Oracle quedó fuera por el enunciado, SQL Server Express tiene límites de tamaño y PostgreSQL exigía aprender otro lenguaje de procedimientos.
Lógica de negocio dentro de MySQL	Retener, pagar y reembolsar créditos son operaciones que no pueden quedar a medias. Al hacerlas en procedimientos con START TRANSACTION, COMMIT y ROLLBACK, la atomicidad no depende del backend.
Persistencia políglota con MongoDB	Publicaciones, chat y bitácora tienen mucho volumen, estructura variable y no requieren transacciones entre entidades. MongoDB permite embeber comentarios, agregar campos sin migraciones y resolver estadísticas con su framework de agregación (Sadalage y Fowler, 2012).
Un documento por mensaje de chat	Evita documentos que crecen sin límite y permite consultar por misión y fecha con un índice compuesto.
Ubuntu Server 26.04 LTS en VirtualBox	El enunciado exige Linux o Unix. Ubuntu Server tiene soporte extendido y paquetes para ambos motores. VirtualBox es gratuito y funciona en los equipos de los tres integrantes.
Dos usuarios de base de datos por motor	bountyline_app, que usa el backend, solo tiene SELECT y EXECUTE en MySQL y readWrite en MongoDB. bountyline_admin queda para administración. Las contraseñas se pasan por variables de entorno.

Scrum con tablero Kanban	El trabajo se dividió en sprints de dos semanas alineados con las dos fases de entrega, con el backlog priorizado con MoSCoW (Schwaber y Sutherland, 2020).

5. Aplicación de los temas del curso

El proyecto se construyó sobre los tres temas vistos en el semestre: SQL avanzado, PL/SQL y MQL. Como el enunciado no permite Oracle, la parte de PL/SQL se implementó con el lenguaje procedimental de MySQL, que ofrece los mismos mecanismos con otra sintaxis.

Tema	Cómo se aplicó en BountyLine
SQL avanzado	Las 12 consultas del documento SQL de reportes combinan JOIN entre varias tablas, GROUP BY con funciones de agregación (COUNT, SUM, AVG) y expresiones CASE. Las 3 vistas encapsulan consultas que usan el backend y los reportes. El modelo usa restricciones CHECK, llaves foráneas y SELECT ... FOR UPDATE para bloquear filas durante las operaciones con créditos.
PL/SQL (equivalente en MySQL)	24 procedimientos, 4 funciones y 14 triggers. Los bloques EXCEPTION WHEN de PL/SQL se traducen en DECLARE EXIT HANDLER con ROLLBACK y RESIGNAL, y RAISE_APPLICATION_ERROR se reemplaza por SIGNAL SQLSTATE '45000' con un mensaje propio. Las operaciones de créditos usan START TRANSACTION, COMMIT y ROLLBACK.
MQL	CRUD con insertOne, insertMany, find con filtros y proyecciones, updateOne, updateMany y deleteOne. Búsqueda por texto con un índice $text. Pipelines de agregación con $match, $unwind, $group, $project, $sort y $lookup para estadísticas del feed, del chat y de la bitácora.

6. Resultados

El modelo relacional tiene 14 entidades normalizadas hasta tercera forma normal, con relaciones 1:1, 1:N y N:M y una especialización total y disjunta de Cuenta en Usuario, Cazarrecompensas y Administrador. Se documentó con el modelo Entidad-Relación de Peter Chen, el modelo extendido, el modelo relacional en notación Crow's Foot y el diccionario de datos.

En MySQL se implementaron 4 funciones, 24 procedimientos almacenados, 14 triggers y 3 vistas, con manejo de excepciones y control de transacciones en las operaciones de créditos. La carga de prueba deja 24 cuentas, 24 misiones, 68 postulaciones, 61 movimientos, 9 calificaciones y 4 reportes. El documento SQL de reportes tiene 12 consultas para el administrador, entre ellas misiones por zona, rendimiento de cazarrecompensas, movimientos de créditos por tipo y misiones cerradas por mes.

En MongoDB se crearon las colecciones publicaciones, mensajes_chat y bitacora_actividad con validadores $jsonSchema e índices, y se cargaron 28 publicaciones, 96 mensajes y 160 eventos. Las consultas de agregación usan $match, $unwind, $group, $sort y $lookup para obtener, por ejemplo, las etiquetas más usadas por zona y la actividad por semana.

La aplicación web se conecta a los dos motores y cubre los 15 requerimientos funcionales y 7 no funcionales definidos en la propuesta, con su trazabilidad hacia los objetos de base de datos y 22 criterios de aceptación. En la parte de gestión quedaron el acta de constitución con el diagrama de Gantt, la matriz RACI, la matriz de 7 riesgos con su mitigación, el plan de recuperación ante desastres, la política de backups y el acta de cierre.

7. Lecciones aprendidas

- Pasar de PL/SQL en Oracle al lenguaje procedimental de MySQL obligó a entender los conceptos más allá de la sintaxis. El manejo de excepciones, las transacciones y los triggers funcionan con la misma lógica, pero cambian la forma de declarar los manejadores de error y de lanzar mensajes propios.
- Poner la lógica crítica en procedimientos almacenados simplificó el backend y permitió quitarle al usuario de la aplicación los permisos de INSERT, UPDATE y DELETE. Conviene definir esos permisos desde el diseño y no al final.
- La documentación se desactualiza rápido si no se revisa contra el código. Al cerrar el proyecto encontramos versiones de software que ya no correspondían y consultas de reportes con nombres de columnas distintos a los del script. Revisar los documentos contra los scripts antes de cada entrega habría evitado ese trabajo extra.
- Decidir qué datos van a MongoDB con criterios claros (volumen, estructura variable y ausencia de transacciones) evitó que la base documental terminara siendo una copia del modelo relacional.
- Automatizar la instalación de las máquinas virtuales y de los motores con scripts hizo que el entorno se pudiera repetir en otro equipo sin depender de pasos manuales.
- Las credenciales nunca deben quedar escritas en el repositorio. Desde el inicio es mejor leerlas de variables de entorno.

8. Conclusiones

BountyLine cumple el objetivo del curso: integra una base de datos relacional con lógica de negocio implementada en el motor, una base de datos documental en MongoDB y una aplicación web que se conecta a las dos, todo instalado en máquinas virtuales Linux y gestionado como un proyecto de TI. En el camino se aplicaron los tres temas del semestre: SQL avanzado en los reportes y las vistas, la lógica procedimental equivalente a PL/SQL en procedimientos, funciones y triggers, y MQL en las operaciones y agregaciones de MongoDB.

La combinación de MySQL y MongoDB mostró que cada motor rinde mejor con el tipo de dato para el que fue pensado. Las operaciones con créditos quedaron protegidas por transacciones y validaciones dentro de la base, mientras que el contenido social ganó flexibilidad y consultas de agregación sencillas. Como trabajo futuro quedan pruebas de carga sobre el feed, notificaciones en tiempo real y un despliegue fuera del entorno de laboratorio.

Referencias

Chen, P. P. (1976). The entity-relationship model: Toward a unified view of data. ACM Transactions on Database Systems, 1(1), 9-36. https://doi.org/10.1145/320434.320440

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377-387. https://doi.org/10.1145/362384.362685

Elmasri, R. y Navathe, S. B. (2016). Fundamentals of database systems (7.ª ed.). Pearson.

Bradshaw, S., Brazil, E. y Chodorow, K. (2019). MongoDB: The definitive guide (3.ª ed.). O'Reilly Media.

Sadalage, P. J. y Fowler, M. (2012). NoSQL distilled: A brief guide to the emerging world of polyglot persistence. Addison-Wesley.

Schwaber, K. y Sutherland, J. (2020). La guía de Scrum. https://scrumguides.org

Smith, J. E. y Nair, R. (2005). Virtual machines: Versatile platforms for systems and processes. Morgan Kaufmann.

Anexos

- Fase 1: propuesta, documento de supuestos, modelo E-R de Peter Chen, modelo E-R extendido, modelo relacional Crow's Foot, diccionario de datos y documento SQL de reportes.
- Fase 2: análisis de selección de SO, implementación de la VM, análisis de selección de RDBMS, implementación del RDBMS, scripts SQL, modelo documental MongoDB, implementación MongoDB y documentos de conexión de la aplicación.
- Gestión: acta de constitución, plan de recuperación ante desastres, política de backups, acta de cierre y diagrama C4 de componentes.
- Repositorio Git del proyecto y video demostrativo.